

同的业务模型以多种方式保持不变，让应用程序更加灵活。持久性选择可能会随着时间的推移而发生变化，从一种数据库技术变为另一种数据库技术，或除应用程序一开始具备的持久性形式之外还需要其他形式的持久性（例如，除相关数据库之外还需使用 Redis 缓存或 Azure Cosmos DB）。

违反此原则的一些示例如下：

- (1) 必需的基类。
- (2) 必需的接口实现。
- (3) 负责保存其自身的类（如活动记录模式）。
- (4) 所需的无参数构造函数。
- (5) 需要 `virtual` 关键字的属性。
- (6) 特定于持久性的必需特性。

要求类具有上述任何特性或行为，会增加要保持不变的类型和持久性技术的选择之间的耦合，从而增加将来采用新的数据访问策略的难度。

8. 有界上下文

有界上下文是领域驱动设计中的中心模式。它们可以将大型应用程序或组织分解为独立的概念模块，通过这种方式来解决复杂性问题。每个概念模块表示各自独立的上下文（因此有界），并且可以独立改进。理想情况下，每个有界上下文都能够为其中的概念自由选择它自己的名称，并对其自己的持久性存储具有独占访问权限。

至少，各 Web 应用程序应努力成为自己的有界上下文，为其业务模型提供自己的持久性存储，而不是与其他应用程序共享数据库。有界上下文之间的通信通过编程接口进行，而不是通过共享数据库进行，这样可以引发业务逻辑和事件来响应发生的更改。有界上下文会紧密映射到微服务，后者在理想情况下也作为其自己的单独有界上下文实现。

16.2.2 Web应用架构分类

Web 应用程序的体系结构可以根据软件开发和部署模式分为不同的类别。

1. 整体式架构

整体式架构是一种传统的软件开发模型（也称为 Web 开发架构），其中整个软件作为一段代码开发，运用传统的瀑布模型。这意味着所有组件都是相互依赖和互连的，并且每个组件都需要运行应用程序。若要更改或更新特定功能，需要重写并编译整个代码。

由于整体架构将整个代码视为单个程序，因此构建新项目、应用框架、脚本和模板并对其进行测试变得简单易行，并且部署也很容易。但是，随着代码量越来越大，管理或进行更新变得困难；即使是很小的更改，也需要经历 Web 开发架构的整个过程。由于每个元素都是相互依赖的，因此扩展应用程序并不容易。而且它并不可靠，因为单点故障可能会使应用程序瘫痪。当开发人员想要构建轻量级应用程序并且预算紧张时，整体式架构将达到这一目的。但是，当开发团队在同一地点工作而不是远程分布时，使用整体模型是有意义的。

2. 微服务架构

微服务架构解决了在整体环境中遇到的几个挑战。在微服务架构中，代码被开发为松散耦合的独立服务，通过 RESTful API 进行通信。每个微服务都包含自己的数据库并运行特定的业务逻辑，这意味着您可以轻松开发和部署独立的服务。

由于它是松散耦合的，微服务架构提供了更新/修改和扩展独立服务的灵活性。开发变得简单高效，并实现持续交付。开发人员可以快速适应创新。对于高度可扩展和复杂的应用程序，微服务是一个不错的选择。但是，使用运行时实例部署多个服务是一项挑战。当服务数量增加时，管理它们的复杂性也会增加。此外，微服务需应用共享分区数据库。这意味着应确保受事务影响的多个数据库之间的一致性。

3. 无服务器架构

无服务器架构是开发软件应用程序的模型。在此结构中，底层基础结构的预配由基础结构服务提供商管理。这意味着用户只需为正在使用的基础架构付费，而无须为空闲 CPU 时间或未使用的空间付费。无服务器计算降低了成本，因为资源仅在应用程序执行时被使用。扩展任务由云提供商处理。此外，后端代码得到简化。它减少了开发工作和成本，并加快了上市时间。多媒体处理、实时流媒体、聊天机器人 CI 管道、物联网传感器消息等，是无服务器计算的一些用例。在微服务架构中，可以使用 AWS Lambda、API Gateway 和 API Step Functions 执行无服务器计算。

在构建 Web 应用系统架构时，应根据系统需求与开发资源条件选择 Web 应用系统结构类型，图 16-3 详细展示了整体式架构和微服务架构的具体结构与区别。

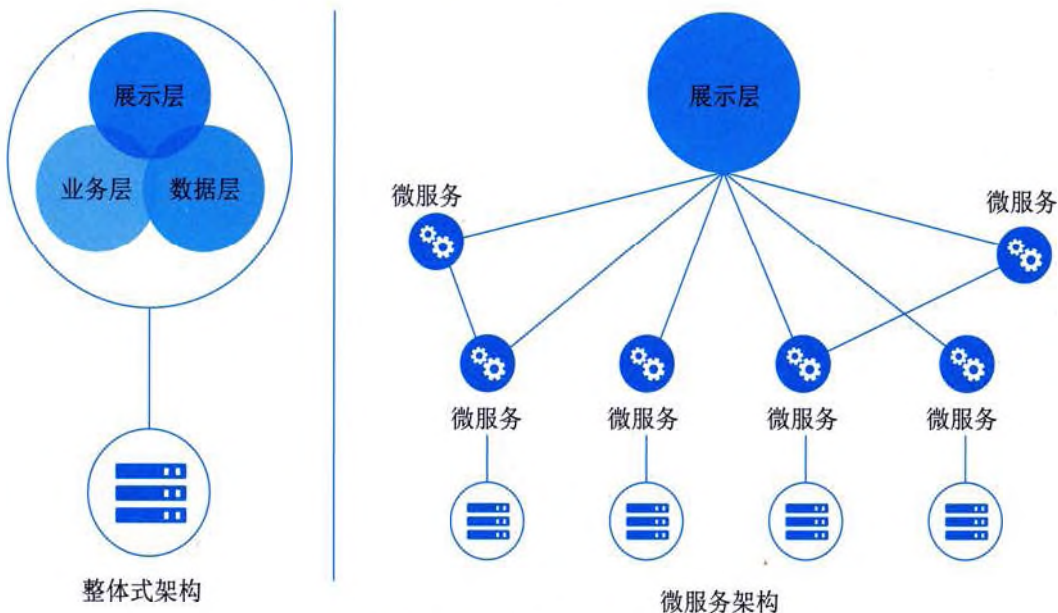


图 16-3 整体式架构和微服务架构